

BRACT's

Vishwakarma Institute of Technology, Pune

Practical Implementation Sheet

Department: CSE-AI Semester: 5 Academic Year: 2026-27

Division: E Batch: 2 Practical No: 4

Roll no: 57 PRN No:12410559 Name of Student: Avni Selote

Subject Name : Deep Learning

Problem Statement :

Develop an LSTM-based model for time-series forecasting using stock price, weather, or sales datasets.

Algorithm :

1. Load and preprocess the weather dataset.
2. Sort data by Date/Time and select relevant features.
3. Split data into 80% training and 20% testing.
4. Normalize the data using Min-Max Scaling.
5. Create sequences of the previous 24 hours.
6. Build and train the LSTM model.
7. Predict temperature on test data.
8. Calculate MAE and RMSE.
9. Plot actual vs predicted temperature.

Code & Output :

```
#Problem Statement :Develop an LSTM-based model for time-series forecasting using stock price, weather, or sales datasets.
#Importing the required libraries
import os
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.preprocessing import MinMaxScaler
from sklearn.metrics import mean_absolute_error, mean_squared_error

#LSTM to process time series data , Dropout-Regularization technique to prevent overfitting

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import LSTM, Dense, Dropout

#Importing the dataset directly from kaggle
import kagglehub

# Download latest version
path = kagglehub.dataset_download("vidhisrivastava/weather-dataset")

print("Path to dataset files:", path)

Downloading to /root/.cache/kagglehub/datasets/vidhisrivastava/weather-dataset/1.archive...
100% 101k/101k [00:00<00, 304kB/s]Extracting files...
Path to dataset files: /root/.cache/kagglehub/datasets/vidhisrivastava/weather-dataset/versions/1
```

```
import os
```

```
print(os.listdir(path))
```

```
['Project1WeatherDataset.csv']
```



```
# Load the CSV file into a Pandas DataFrame
```

```
file_path = os.path.join(path, "Project1WeatherDataset.csv")
```

```
df = pd.read_csv(file_path)
```

```
print(df.head())
```

```
print(df.shape)
```

```
print(df.columns)
```

```
print(df.info())
```

```
...      Date/Time  Temp_C  Dew Point  Temp_C  Rel Hum_%  Wind Speed_kmh  \
0  1/1/2012 0:00    -1.8         -3.9    -3.9      86         4
1  1/1/2012 1:00    -1.8         -3.7    -3.7      87         4
2  1/1/2012 2:00    -1.8         -3.4    -3.4      89         7
3  1/1/2012 3:00    -1.5         -3.2    -3.2      88         6
4  1/1/2012 4:00    -1.5         -3.3    -3.3      88         7
```

```
      Visibility_km  Press_kPa      Weather
```

```
0          8.0     101.24         Fog
```

```
1          8.0     101.24         Fog
```

```
2          4.0     101.26  Freezing Drizzle,Fog
```

```
3          4.0     101.27  Freezing Drizzle,Fog
```

```
# Convert the Date/Time column into datetime format and sort the data
```

```
df["Date/Time"] = pd.to_datetime(df["Date/Time"], errors="coerce")
```

```
df = df.sort_values("Date/Time")
```

```
df = df.dropna(subset=["Date/Time"])
```

```
print(df[["Date/Time"]].head())
```

```
      Date/Time
```

```
0  2012-01-01 00:00:00
```

```
1  2012-01-01 01:00:00
```

```
2  2012-01-01 02:00:00
```

```
3  2012-01-01 03:00:00
```

```
4  2012-01-01 04:00:00
```

```
df = df.sort_values("Date/Time")
```

```
# Select numerical features
```

```
features = [  
    "Temp_C",  
    "Dew Point Temp_C",  
    "Rel Hum_%",  
    "Wind Speed_km/h",  
    "Visibility_km",  
    "Press_kPa"  
]
```

```
df = df[["Date/Time"] + features]
```

```
# Remove missing values
```

```
df = df.dropna()
```

```
print(df.head())
```

```
print(df.shape)
```

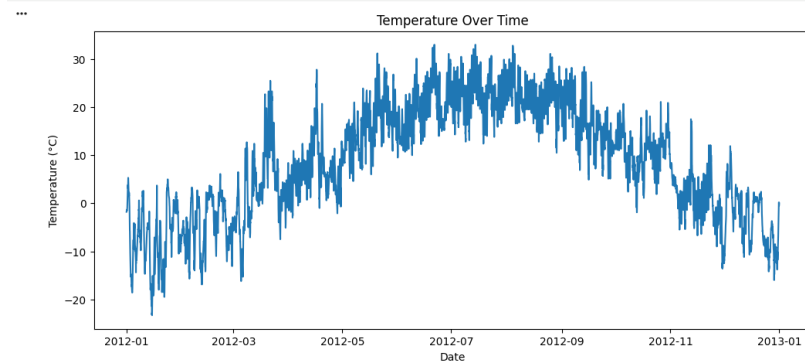
```
...      Date/Time  Temp_C  Dew Point Temp_C  Rel Hum_%  Wind Speed_km/h  \  
0 2012-01-01 00:00:00   -1.8             -3.9         86           4  
1 2012-01-01 01:00:00   -1.8             -3.7         87           4  
2 2012-01-01 02:00:00   -1.8             -3.4         89           7  
3 2012-01-01 03:00:00   -1.5             -3.2         88           6  
4 2012-01-01 04:00:00   -1.5             -3.3         88           7  
  
      Visibility_km  Press_kPa  
0                8.0      101.24  
1                8.0      101.24
```

```
# Plot temperature over time to observe the time-series pattern  
plt.figure(figsize=(12,5))
```

```
plt.plot(df["Date/Time"], df["Temp_C"])
```

```
plt.xlabel("Date")  
plt.ylabel("Temperature (°C)")  
plt.title("Temperature Over Time")
```

```
plt.show()
```



```
# Split the time-series data
train_size = int(len(df) * 0.8)
```

```
train = df.iloc[:train_size]
test = df.iloc[train_size:]
```

```
print("Training samples:", len(train))
print("Testing samples:", len(test))
```

```
Training samples: 7027
Testing samples: 1757
```

```
#Normalize the weather features to a range between 0 and 1
scaler = MinMaxScaler()
```

```
train_scaled = scaler.fit_transform(train[features])
test_scaled = scaler.transform(test[features])
```

```
▶ # Create sequences of the previous 24 hours to predict the temperature of the next hour
time_steps = 24
```

```
def create_sequences(data, time_steps):
    X = []
    y = []

    for i in range(time_steps, len(data)):
        X.append(data[i-time_steps:i])
        y.append(data[i, 0]) # Temp_C is first column

    return np.array(X), np.array(y)

X_train, y_train = create_sequences(train_scaled, time_steps)
X_test, y_test = create_sequences(test_scaled, time_steps)

print("X_train shape:", X_train.shape)
print("y_train shape:", y_train.shape)
print("X_test shape:", X_test.shape)
print("y_test shape:", y_test.shape)
```

```
... X_train shape: (7003, 24, 6)
    y_train shape: (7003,)
    X_test shape: (1733, 24, 6)
    y_test shape: (1733,)
```

```
# Build an LSTM neural network
# dropout layers, and dense layers for temperature prediction
model = Sequential()

model.add(
    LSTM(
        64,
        return_sequences=True,
        input_shape=(X_train.shape[1], X_train.shape[2])
    )
)

model.add(Dropout(0.2))

model.add(LSTM(32))

model.add(Dropout(0.2))

model.add(Dense(16, activation="relu"))

model.add(Dense(1))

model.compile(
    optimizer="adam",
    loss="mse"
)

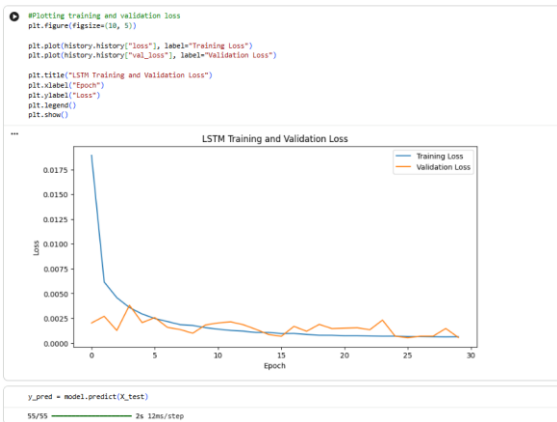
model.summary()
```

... /usr/local/lib/python3.13/dist-packages/keras/src/layers/rnn/rnn.py:199: UserWarning:
super().__init__(**kwargs)
Model: "sequential_2"

Layer (type)	Output Shape	Param #
lstm_4 (LSTM)	(None, 24, 64)	18,176
dropout_4 (Dropout)	(None, 24, 64)	0
lstm_5 (LSTM)	(None, 32)	12,416
dropout_5 (Dropout)	(None, 32)	0
dense_3 (Dense)	(None, 16)	528
dense_4 (Dense)	(None, 1)	17

Total params: 31,137 (121.63 KB)
Trainable params: 31,137 (121.63 KB)
Non-trainable params: 0 (0.00 B)

```
#Training the model
history = model.fit(
    X_train,
    y_train,
    epochs=30,
    batch_size=32,
    validation_split=0.1,
    verbose=1
)
```



```
# Calculate Mean Absolute Error (MAE) and Root Mean Squared Error (RMSE)
# to measure the accuracy of the temperature predictions
```

```
mae = mean_absolute_error(y_test_actual, y_pred_actual)
```

```
rmse = np.sqrt(
    mean_squared_error(y_test_actual, y_pred_actual)
)
```

```
print("MAE:", mae)
print("RMSE:", rmse)
```

```
MAE: 0.9745096877019519
RMSE: 1.2417960887124436
```

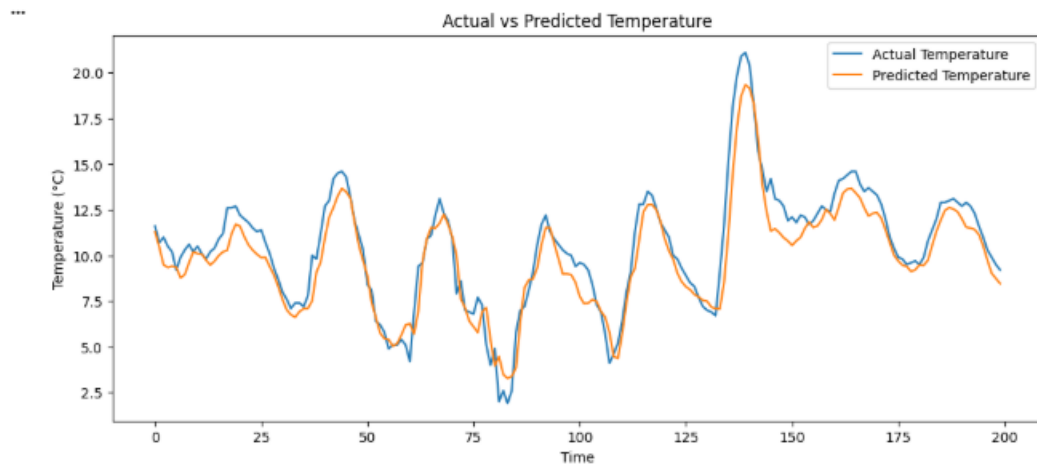
```
▶ #comparing actual vs predicted temperature
plt.figure(figsize=(12,5))
```

```
plt.plot(
    y_test_actual[:200],
    label="Actual Temperature"
)
```

```
plt.plot(
    y_pred_actual[:200],
    label="Predicted Temperature"
)
```

```
plt.xlabel("Time")
plt.ylabel("Temperature (°C)")
plt.title("Actual vs Predicted Temperature")
```

```
plt.legend()
plt.show()
```



```
▶ #predicting the next temperature
last_24_hours = test_scaled[-24:]
```

```
last_24_hours = last_24_hours.reshape(
    1,
    24,
    len(features)
)
```

```
next_temp_scaled = model.predict(last_24_hours)
```

```
next_temp = (
    next_temp_scaled[0][0] *
    (temp_max - temp_min) +
    temp_min
)
```

```
print("Predicted next temperature:", round(next_temp, 2), "°C")
```

```
... 1/1 0s 46ms/step
Predicted next temperature: 0.33 °C
```